

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5 – REKURSIF



NAMA : Yazid Yaskur Muhammad

NIM : 109082500078

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Rekursif adalah teknik dalam pemrograman di mana suatu fungsi atau prosedur memanggil dirinya sendiri untuk menyelesaikan masalah dengan cara memecahnya menjadi sub-masalah yang lebih kecil namun sejenis. Dalam rekursif terdapat dua komponen utama, yaitu base case sebagai kondisi untuk menghentikan proses rekursi dan recursive case sebagai bagian yang melakukan pemanggilan fungsi itu sendiri. Proses rekursif berlangsung melalui dua tahap, yaitu forward (pemanggilan fungsi secara berulang hingga mencapai base case) dan backward (kembali ke pemanggilan sebelumnya setelah kondisi dasar terpenuhi). Teknik ini dapat digunakan sebagai alternatif dari perulangan (looping), dan setiap algoritma rekursif pada dasarnya dapat diubah menjadi bentuk iteratif.

B. Guided

1. Guided 1

```
C. package main
D. import "fmt"
E.
F. func main() {
G.     var n int
H.     fmt.Scan(&n)
I.     baris(n)
J. }
K. func baris(bilangan int) {
L.     if bilangan == 1 {
M.         fmt.Println(1)
N.     } else {
O.         fmt.Println(bilangan)
P.         baris(bilangan - 1)
Q.     }
```

Output :

```
PS C:\Users\Asus\Documents\AlproPraktikum\
Yaskur-Muhammad\Week 5 - Modul 5\Guided\Gu
1
1
PS C:\Users\Asus\Documents\AlproPraktikum\
Yaskur-Muhammad\Week 5 - Modul 5\Guided\Gu
5
5
4
3
2
1
```

2. Guided 2

```
package main
import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output :

```
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yaskur-Muhammad\Week 5 - Modul 5\Guided\Guided 2\Guided2
4
10
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yaskur-Muhammad\Week 5 - Modul 5\Guided\Guided 2\Guided2
5
15
```

3. Guided 3

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
```

```

        return 1
    } else {
        return 2 * pangkat(n-1)
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-
Yaskur-Muhammad\Week 5 - Modul 5\Guided\Guided 3\Guided3.go"
5
32
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-
Yaskur-Muhammad\Week 5 - Modul 5\Guided\Guided 3\Guided3.go"
3
8

```

C. Unguided

1. Soal Unguided 1

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan tertentu.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan dari N hingga 1 dan kembali ke N.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1.	5	5 4 3 2 1 2 3 4 5
2.	9	9 8 7 6 5 4 3 2 1 2 3 4 5 6 7 8 9

Jawaban :

```

package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }
}

```

```

    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    var n int
    fmt.Print("Masukkan jumlah suku: ")
    fmt.Scan(&n)

    fmt.Println("Deret Fibonacci:")
    for i := 0; i < n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\10908\
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Ungui
Masukkan jumlah suku: 4
Deret Fibonacci:
0 1 1 2
PS C:\Users\Asus\Documents\AlproPraktikum\10908\
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Ungui
Masukkan jumlah suku: 4
Deret Fibonacci:
0 1 1 2

```

Penjelasan :

Program tersebut digunakan untuk menampilkan deret Fibonacci sebanyak jumlah yang diinput oleh pengguna dengan memanfaatkan fungsi rekursif. Fungsi `fibonacci(n)` bekerja dengan dua kondisi dasar (base case), yaitu ketika $n = 0$ menghasilkan 0 dan $n = 1$ menghasilkan 1, sedangkan untuk nilai lainnya dihitung dengan menjumlahkan dua nilai sebelumnya, yaitu `fibonacci(n-1) + fibonacci(n-2)`. Pada fungsi utama (main), program meminta input jumlah suku dari pengguna, kemudian menggunakan perulangan untuk memanggil fungsi `fibonacci(i)` dari indeks 0 hingga $n-1$ dan menampilkan hasilnya secara berurutan sehingga terbentuk deret Fibonacci.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1.	5	1 3 5
2.	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```
package main
import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }

    if n%i == 0 {
        fmt.Print(i, " ")
    }

    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Println("Faktor dari", n, "adalah:")
    faktor(n, 1)
}
```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\10
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Un
Masukkan bilangan: 5
Faktor dari 5 adalah:
1 5
PS C:\Users\Asus\Documents\AlproPraktikum\10
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Un
Masukkan bilangan: 12
Faktor dari 12 adalah:
1 2 3 4 6 12

```

Penjelasan :

Program tersebut digunakan untuk menampilkan faktor-faktor dari suatu bilangan yang diinput oleh pengguna dengan menggunakan metode rekursif. Fungsi faktor(n, i) bekerja dengan memeriksa setiap bilangan mulai dari $i = 1$ hingga n , di mana jika $n \% i == 0$ maka i merupakan faktor dari n dan akan ditampilkan. Proses rekursif akan terus berjalan dengan memanggil fungsi faktor(n, i+1) hingga kondisi berhenti (base case) tercapai, yaitu ketika $i > n$. Pada fungsi main, program meminta input bilangan dari pengguna, kemudian memanggil fungsi faktor(n, 1) untuk mulai mencari dan menampilkan semua faktor dari bilangan tersebut secara berurutan.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk mencari hasil pangkat dari dua

buah bilangan.

Masukan terdiri dari bilangan bulat x dan y .

Keluaran terdiri dari hasil x dipangkatkan y .

Catatan: diperbolehkan menggunakan asterik "*", tapi dilarang menggunakan import "math".

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1.	2 2	4
2.	5 3	1 2 5

Jawaban :

```

package main
import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }

    if i%2 != 0 {
        fmt.Print(i, " ")
    }

    ganjil(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Println("Bilangan ganjil dari 1 sampai", n, "adalah:")
    ganjil(n, 1)
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yas
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Unguided 3\Unguided3.go
Masukkan bilangan: 5
Bilangan ganjil dari 1 sampai 5 adalah:
1 3 5
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yas
Yaskur-Muhammad\Week 5 - Modul 5\Unguided\Unguided 3\Unguided3.go
Masukkan bilangan: 20
Bilangan ganjil dari 1 sampai 20 adalah:
1 3 5 7 9 11 13 15 17 19

```

Penjelasan :

Program tersebut digunakan untuk menampilkan bilangan ganjil dari 1 sampai dengan bilangan yang diinput oleh pengguna menggunakan metode rekursif. Fungsi `ganjil(n, i)` bekerja dengan memeriksa setiap nilai `i` mulai dari 1 hingga `n`, di mana jika `i % 2 != 0` maka `i` merupakan bilangan ganjil dan akan ditampilkan. Proses ini dilakukan secara berulang melalui pemanggilan fungsi `ganjil(n, i+1)` hingga mencapai kondisi

berhenti (base case), yaitu ketika $i > n$. Pada fungsi main, program menerima input dari pengguna lalu memanggil fungsi ganjil(n , 1) untuk menampilkan seluruh bilangan ganjil dalam rentang tersebut secara berurutan.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa rekursif merupakan metode yang efektif untuk menyelesaikan permasalahan yang memiliki pola berulang, seperti deret Fibonacci, pencarian faktor bilangan, dan penentuan bilangan ganjil. Setiap program rekursif harus memiliki base-case sebagai kondisi berhenti dan recursive-case sebagai proses pemanggilan ulang fungsi. Tanpa base-case, program akan berjalan terus menerus tanpa henti. Implementasi rekursif juga memperlihatkan bahwa suatu permasalahan dapat diselesaikan dengan cara yang lebih sederhana dan terstruktur, meskipun dari segi efisiensi terkadang lebih lambat dibandingkan metode iteratif. Oleh karena itu, pemahaman konsep dasar rekursif sangat penting agar dapat digunakan secara tepat dalam pemrograman.